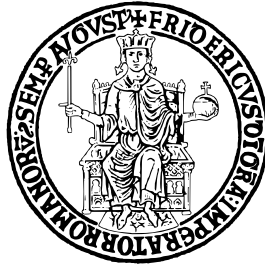


UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

INGEGNERIA DEL SOFTWARE

DOCUMENTAZIONE BUGBOARD26

Anno Accademico 2025–2026

Indice

1	Glossario	4
2	Use case diagrams	6
2.1	Gestione delle ambiguità	6
2.2	UCD	6
3	Requisiti non funzionali	13
3.1	Affidabilità	13
3.2	Performance	13
3.3	Manutenibilità	13
3.4	Sicurezza	13
4	Requisiti di Dominio	14
4.1	Tipologia di Issue	14
4.2	Stati delle Issue	14
4.3	Priorità delle Issue	14
5	Formalizzazione dei casi d'uso significativi	15
5.1	Fully dressed use case	15
5.2	Use case mock-up	16
6	Personas	17
7	Design del sistema	23
7.1	Architettura del sistema	23
7.1.1	Client	23
7.1.2	Server	24
7.2	Scelte tecnologiche adottate	25
7.2.1	Servizi Cloud usati:	25

8	Dockerfile e file di build automatica	26
8.1	Dockerfile	26
8.2	Orchestrazione con Docker Compose	27
8.3	Descrizione dei Servizi Docker Compose	28
9	Design del Software	30
9.1	Schema per la persistenza dei dati	30
9.1.1	Tabella User	30
9.1.2	Tabella Issue	31
9.1.3	Tabella Comment	32
9.1.4	Tabella Change	32
9.1.5	Tabella Attachment	33
9.2	Diagramma delle classi	33
9.3	Scelte di software design	36
9.3.1	Design Pattern adottati	36
9.3.2	Adesione ai principi SOLID	36
10	Strumenti e strategie di versioning	38
10.1	Versioning System	38
10.1.1	Branching Strategy	38
11	Statistiche Repository	39
11.1	Premessa	39
11.2	Analisi dei Contributors	39
11.3	Analisi dei Commit	40
11.4	Analisi delle Linee di Codice	41
12	Report di Qualità del Codice	42

Elenco delle figure

1	Login use case diagram	6
2	Segnalazione issue use case diagram	7
3	Visualizzazione riepilogo issue use case diagram	7
4	Assegnazione issue use case diagram	7
5	Visualizzazione notifiche use case diagram	8
6	Inserimento commenti use case diagram	8
7	Modificare stato di una issue use case diagram	8
8	Visualizzazione dashboard use case diagram	9
9	Esportare elenco delle issue use case diagram	9
10	Modificare una issue use case diagram	9
11	Associare etichetta ad una issue use case diagram	10
12	Cercare una issue use case diagram	10
13	Visualizzazione cronologia modifiche issue use case diagram	10
14	Archiviazione issue use case diagram	11
15	Visualizzazione "readonly" issue e commenti use case diagram	11
16	Contrassegna una issue come duplicato use case diagram	11
17	Visualizzazione report mensile use case diagram	12
18	Imposta scadenze per la risoluzione di issue use case diagram	12
19	Mock-up use case 2	16
20	Diagramma delle classi del back-end	34
21	Diagramma delle classi del front-end	35
22	Screenshot del repository su GitHub	38
23	Screenshot dei Contributors, da GitHub	39
24	Screenshot dei Commits over time, da GitHub	40
25	Screenshot della "Code Frequency", da GitHub	41
26	Report di Qualità da Sonar Qube Cloud	42
27	Esempio di Utilizzo di SonarQube for IDE su IntelliJ IDEA	42

1 Glossario

Termine	Definizione
Amministratore	Un utente con privilegi speciali che può gestire utenti, progetti e configurazioni della piattaforma.
Archived	L'issue è stata archiviata, questa sarà visualizzabile da un amministratore in una sezione dedicata.
Assegnatario	L'utente responsabile della gestione di una specifica issue.
Bug	Un difetto o un errore nel software che causa un comportamento imprevisto o indesiderato.
Dashboard	La pagina principale della piattaforma che fornisce una panoramica delle attività recenti, delle issue assegnate e dei progetti in corso.
Done	L'issue è stata risolta.
Fully Dressed	Descrizione testuale specifica per uno use case, effettuata sfruttando strumenti come il Cockburn template.
In Progress	Un utente sta lavorando alla risoluzione dell'issue.
Issue	Un problema, un compito o una richiesta di funzionalità all'interno di un progetto software.
Mock-up	Modellazione a bassa fedeltà di utilizzo di una specifica funzionalità all'interno dell'applicazione.
Priorità	Un indicatore del livello di importanza di una issue, che aiuta a determinare l'ordine in cui le issue devono essere affrontate.
Requisiti Di Dominio	Requisiti derivati dal contesto nel quale il software è usato.
Requisiti Funzionali	Servizi che il sistema dovrebbe fornire e comportamento del sistema di fronte a particolari input in determinate situazioni.
Requisiti Non Funzionali	Vincoli sul sistema e sulle funzioni offerte da esso.

Ruolo	Un insieme di permessi assegnati a un utente che determina le azioni che può eseguire sulla piattaforma.
Stato	Lo stato attuale di una issue, come "To Do", "In Progress", "Done" o "Archived".
Tag	Parole chiave assegnate alle issue per facilitare la categorizzazione e la ricerca.
To Do	L'issue è presente in lista e ancora nessuno ha iniziato a lavorarci.
UCD	Use Case Diagram. Un diagramma facente parte di UML, si compone di Attori e Use Case.
Use Case	Modella un requisito funzionale, corrisponde ad una funzionalità del sistema che porta un beneficio o ha un utilizzo per un attore.
Utente	Una persona che utilizza la piattaforma BugBoard26 per gestire progetti e issue.
Utente Esterno	Un utente esterno ha solo privilegi di lettura.

2 Use case diagrams

In questa sezione sono riportati gli use case diagram per tutti i casi d'uso dell'applicazione identificati a partire dalle funzionalità richieste.

2.1 Gestione delle ambiguità

All'interno della descrizione delle funzionalità da implementare sono stati utilizzati i termini bug e issue spesso in maniera confusoria. L'ambiguità è stata risolta sviluppando le funzionalità per tutti i tipi di issue e non per i bug nello specifico.

La funzionalità di notifica era citata in entrambe le funzionalità 4 e 6, quindi è stato creato uno use case ⁵ a sè stante. La sezione della funzionalità 4, relativa all'invio di una notifica ad un utente quando gli viene assegnata una issue, è stata inclusa nello use case ⁴.

La funzionalità 14, relativa all'assegnazione delle issue in base ai suggerimenti del sistema, è stata resa con un commento all'interno dello use case ⁴.

La funzionalità 15 prevede l'esistenza di "utenti esterni", è stato quindi necessaria l'implementazione di una funzionalità di login per questi espressa nello use case ¹.

2.2 UCD

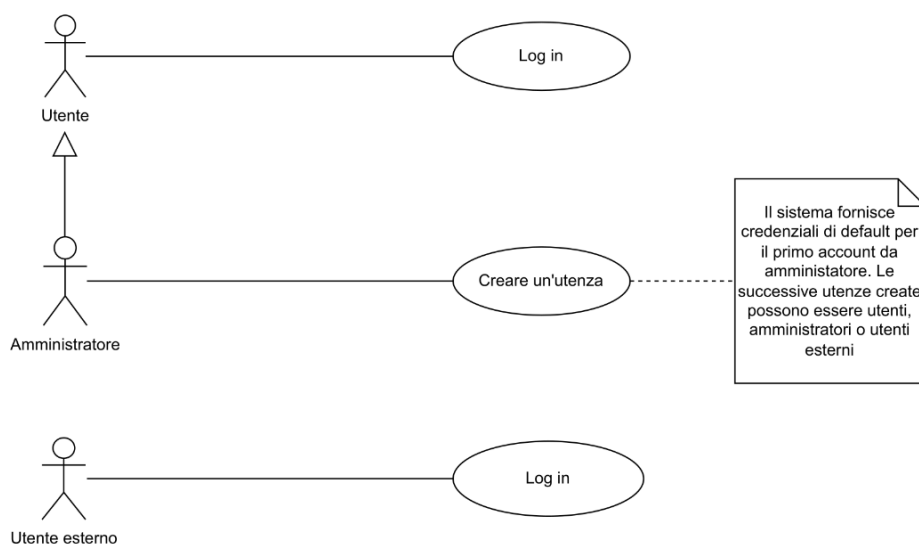


Figura 1: Login use case diagram

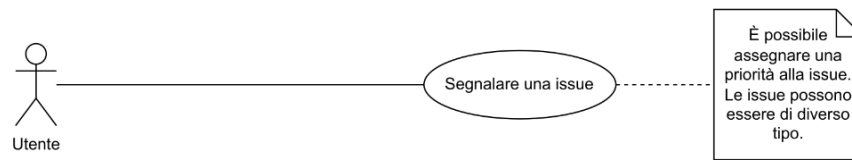


Figura 2: Segnalazione issue use case diagram

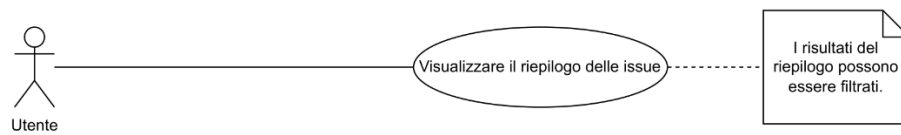


Figura 3: Visualizzazione riepilogo issue use case diagram

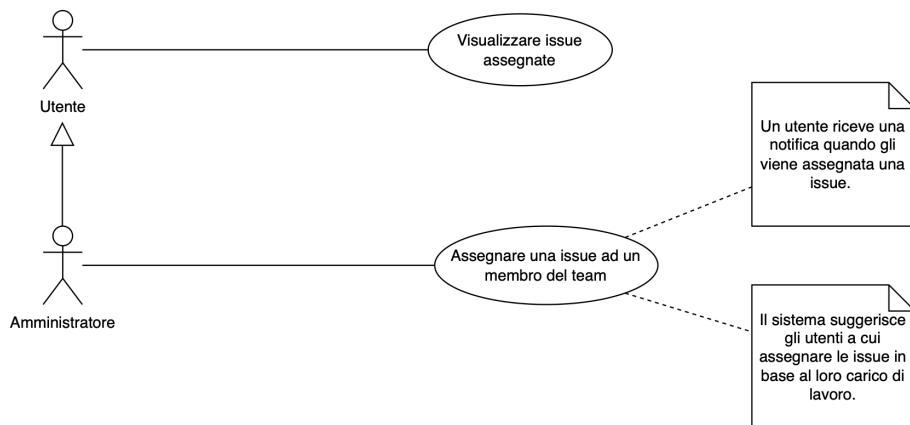


Figura 4: Assegnazione issue use case diagram



Figura 5: Visualizzazione notifiche use case diagram

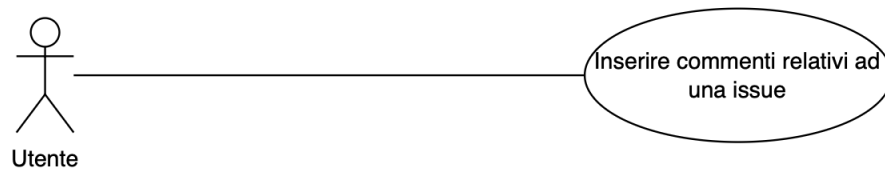


Figura 6: Inserimento commenti use case diagram

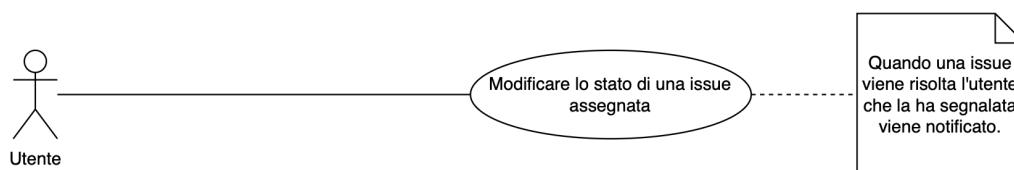


Figura 7: Modificare stato di una issue use case diagram



Figura 8: Visualizzazione dashboard use case diagram

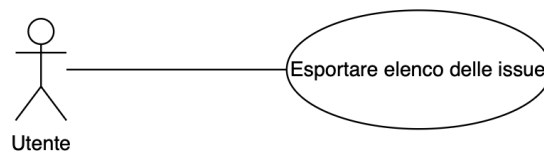


Figura 9: Esportare elenco delle issue use case diagram

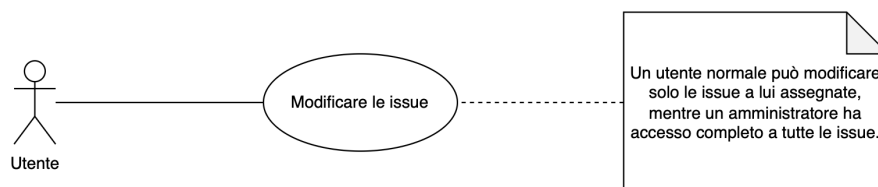


Figura 10: Modificare una issue use case diagram

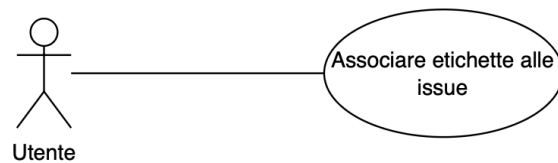


Figura 11: Associare etichetta ad una issue use case diagram

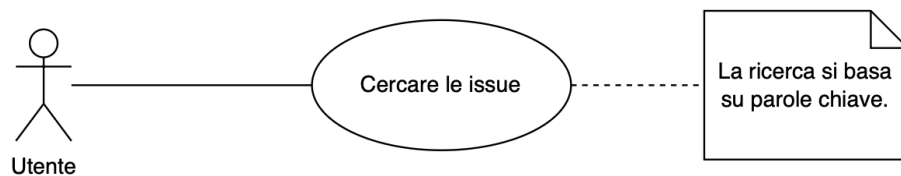


Figura 12: Cercare una issue use case diagram

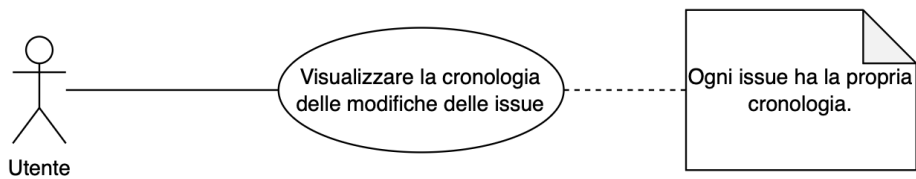


Figura 13: Visualizzazione cronologia modifiche issue use case diagram

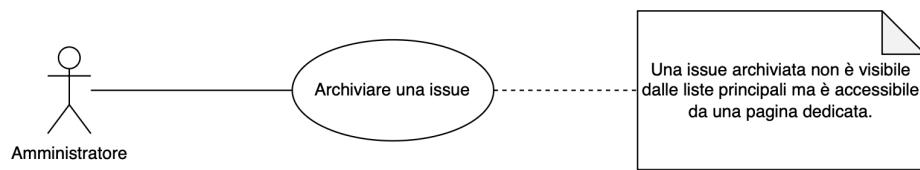


Figura 14: Archiviazione issue use case diagram



Figura 15: Visualizzazione "readonly" issue e commenti use case diagram



Figura 16: Contrassegna una issue come duplicato use case diagram



Figura 17: Visualizzazione report mensile use case diagram

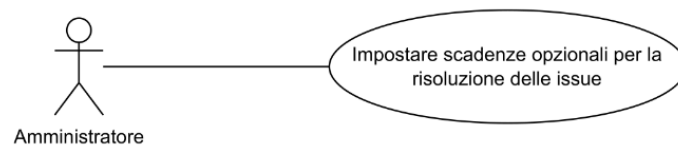


Figura 18: Imposta scadenze per la risoluzione di issue use case diagram

3 Requisiti non funzionali

In questa sezione sono elencati i requisiti non funzionali del sistema BugBoard26, ovvero le caratteristiche che il sistema deve possedere per garantire un'esperienza utente ottimale e un funzionamento efficiente.

3.1 Affidabilità

- **Disponibilità:** Il sistema deve essere disponibile il 99.9% del tempo, con tempi di inattività minimi per manutenzione e aggiornamenti.
- **Recupero da guasti:** In caso di guasto, il sistema deve essere in grado di recuperare rapidamente e ripristinare i dati senza perdita significativa.
- **Gestione dei dati:** Tutti i dati (bug, commenti, cronologie) devono essere salvati in modo persistente lato back-end.

3.2 Performance

- **Tempo di risposta:** Le operazioni principali (come il caricamento della dashboard, la segnalazione di un bug, l'assegnazione di un bug) devono avere un tempo di risposta inferiore a 2 secondi.
- **Scalabilità:** Il sistema deve essere in grado di gestire un aumento del numero di utenti e dati senza degradare le prestazioni, fino a un massimo di 1000 utenti simultanei.

3.3 Manutenibilità

- **Linguaggio usato:** Il codice deve essere strutturato secondo principi di programmazione Object-Oriented (OOP).
- **Modularità:** Il sistema deve essere modulare e separato tra front-end e back-end (architettura distribuita).

3.4 Sicurezza

- **Autenticazione:** Il sistema deve garantire un meccanismo di autenticazione sicuro basato su email e password. Le password devono essere salvate in forma cifrata.

4 Requisiti di Dominio

In questa sezione sono elencati i requisiti di dominio del sistema BugBoard26, ovvero le specifiche e le caratteristiche che il sistema deve soddisfare in relazione al contesto in cui opera.

4.1 Tipologia di Issue

- Ogni issue appartiene a una categoria: bug, feature, question, documentation.

4.2 Stati delle Issue

- Tutte le issue hanno uno stato che può assumere valori come: todo, in progress, done, archived.

4.3 Priorità delle Issue

- Ogni issue ha una priorità che può essere: alta, media, bassa.

5 Formalizzazione dei casi d'uso significativi

In questa sezione sono riportati i casi d'uso significativi dell'applicazione con la loro formalizzazione secondo lo standard Cockburn. In particolare viene riportato il caso d'uso riguardante la segnalazione di una nuova issue.

5.1 Fully dressed use case

Use Case #2	Creazione Issue		
Goal In Context	L'utente vuole creare una nuova issue		
Preconditions	Avere ruolo di amministratore o utente		
Success End Conditions	L'issue viene salvata e aggiunta alla lista delle issue		
Failed End Conditions	L'issue non viene salvata		
Primary Actor	Amministratore/utente		
Trigger	L'utente si autentica e clicca il pulsante "Crea issue"		
Main Scenario	Step	User Action	System
	1	Clicca il bottone "Crea issue"	
	2		Mostra "Creazione Issue"
	3	Compila i campi	
	4		Mostra "Successo"
Extension	Step	User Action	System
	3a<Non compila tutti i campi>	Compila i campi	
	4a		Mostra "Errore"
	5a	Clicca "OK"	
	6a		Mostra "Creazione Issue"
Open Issues	Nessuna		

5.2 Use case mock-up

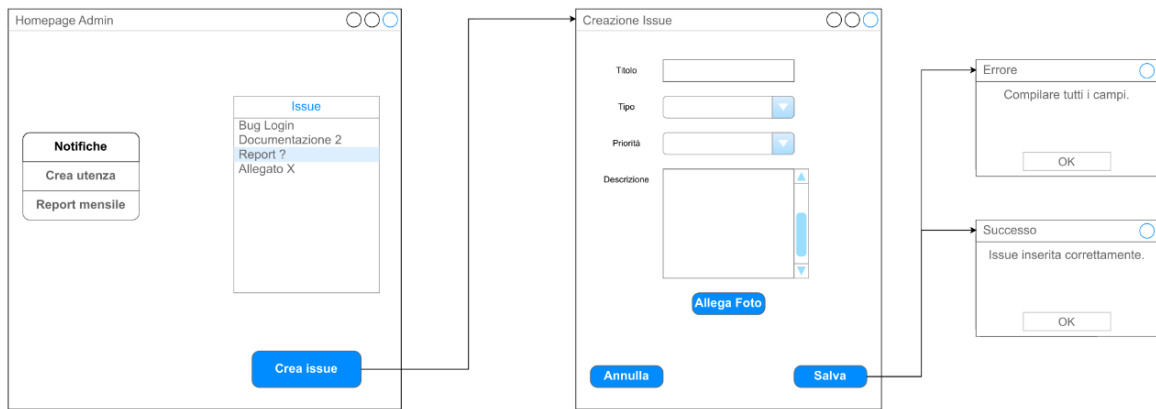


Figura 19: Mock-up use case 2

6 Personas

In questa sezione sono riportate le personas individuate per il progetto. Ognuna di esse rappresenta un utente tipo che potrebbe utilizzare l'applicazione. Sono state individuate sei personas, suddivise in tre categorie: amministratori, utenti e utenti esterni.

Laura Ricci

Età: 24

Ubicazione: Catania, Italia

Stato civile: single

Titolo professionale: Mobile App Developer

Ruolo: Utente

Tratti personali: curiosa, collaborativa, precisa

Obiettivi:

- Segnalare bug in modo rapido e chiaro.
- Collaborare con altri sviluppatori in tempo reale.
- Ricevere notifiche sugli aggiornamenti.

Interessi:

- Architetture distribuite.
- Strumenti collaborativi per il lavoro in team.
- Sviluppo mobile (Android/Kotlin).

Bio:

Laura è una ragazza capace e disponibile che ha trovato lavoro come junior developer dopo aver conseguito la laurea in Informatica con il massimo dei voti. Specializzatasi successivamente nello sviluppo di app mobile, lavora in un piccolo team di ragazzi capaci e volenterosi di collaborare con la propria utenza per migliorare i propri prodotti.

Christian Ranavolo

Età: 21

Ubicazione: Napoli, Italia

Stato civile: fidanzato

Titolo professionale: Junior Developer

Ruolo: Utente

Tratti personali: arrendevole, riservato, scorbutico

Obiettivi:

- Monitorare bug delle app in sviluppo.
- Tenere traccia delle proprie attività.
- Filtrare rapidamente per priorità e stato.

Interessi:

- Tecnologie web moderne.
- Game Development.
- Futsal.

Bio:

Christian esercita la professione di Junior Developer da circa due anni, ossia da quando ha trovato lavoro presso una società di cyber security. Possiede il titolo di “perito informatico” che ha ottenuto col diploma di Istituto Tecnico. Christian non ama il suo lavoro, ma per il momento si accontenta del suo impiego, convinto che possa permettergli di acquisire esperienza nella programmazione che un giorno gli sarà utile nello sviluppo del suo gioco indie.

Marzio Masciarelli

Età: 36

Ubicazione: Firenze, Italia

Stato civile: sposato

Titolo professionale: Project Manager

Ruolo: Amministratore

Tratti personali: organizzato, determinato, comunicativo

Obiettivi:

- Assegnare bug agli sviluppatori in base al carico.
- Monitorare il progresso e le scadenze.
- Produrre report mensili per la direzione.

Interessi:

- Arte e scultura.
- Analisi dei dati e metriche di produttività.
- Robotica.

Bio:

Marzio è un amministratore presso un'azienda occupata nello di sviluppo software nell'ambito della digital art. Marzio ha sempre messo la carriera di fronte a ogni cosa nella sua vita e con la sua caparbia ha raggiunto un ruolo di rilievo in azienda. Tutti sul lavoro lo rispettano e seguono con diligenza le sue direttive. Tuttavia, Marzio sente un vuoto nella sua vita personale e da qualche anno coltiva il desiderio di mettere su famiglia.

Walter Severino Torrone

Età: 44

Ubicazione: Napoli, Italia

Stato civile: sposato

Titolo professionale: Lead Developer

Ruolo: Amministratore

Tratti personali: razionale, tecnico, autorevole

Obiettivi:

- Mantenere ordine nella gestione dei bug.
- Verificare la qualità delle issue segnalate.
- Comunicazione trasparente nel team.

Interessi:

- Sistemi informativi multimediali.
- Version control e code review.
- Software testing e code coverage.

Bio:

Walter è il Lead Developer in una grande azienda informatica. Ha ottenuto la sua posizione in seguito allo sviluppo di software commissionato da un importante cliente, in cui ha avuto un ruolo decisivo. Walter, però, continua nel suo lavoro con umiltà, mantenendo un buon rapporto con tutti i membri del team, con i quali ogni mese organizza una serata karaoke.

Laura Greco

Età: 45

Ubicazione: Verona, Italia

Stato civile: sposata

Titolo professionale: Responsabile Qualità

Ruolo: Utente esterno

Tratti personali: attenta, diplomatica, metodica

Obiettivi:

- Monitorare bug relativi al software.
- Validare la qualità del software consegnato.
- Tracciabilità dei problemi aperti.

Interessi:

- Sicurezza Informatica.
- Tennis.
- Politica e attualità.

Bio:

Laura è la Responsabile Qualità di un'azienda cliente che collabora a progetti software con fornitori esterni. Il suo compito è assicurarsi che i prodotti consegnati rispettino gli standard qualitativi e che eventuali problemi vengano tracciati e risolti in modo puntuale. Pur non avendo competenze tecniche avanzate, è molto attenta ai dettagli e predilige strumenti che le permettano di monitorare l'avanzamento dei lavori attraverso interfacce chiare e report sintetici.

Pietro Balzano

Età: 60

Ubicazione: Napoli, Italia

Stato civile: sposato

Titolo professionale: Direttore Tecnico

Ruolo: Utente esterno

Tratti personali: esigente, pragmatico, orientato ai risultati

Obiettivi:

- Ottenere una visione globale dell'avanzamento del progetto.
- Ricevere report periodici sull'attività del team.
- Verificare che le criticità siano risolte in modo tempestivo.

Interessi:

- Moda.
- Astrofisica.
- Disegno tecnico.

Bio:

Pietro è il Direttore Tecnico di un'azienda partner coinvolta nel progetto. Supervisiona più team e si occupa di garantire che le attività procedano nei tempi stabiliti e con la qualità richiesta. Ha bisogno di un software per monitorare l'andamento dei lavori, verificare la risoluzione delle criticità e assicurarsi che il flusso di comunicazione tra i vari reparti rimanga efficiente. Per lui sul lavoro sono fondamentali chiarezza, efficienza e rapidità.

7 Design del sistema

In questo capitolo viene approfondita la progettazione del sistema realizzato. L'obiettivo è documentare le scelte progettuali che hanno guidato la traduzione dei requisiti in un sistema pienamente funzionante.

7.1 Architettura del sistema

Il software sviluppato adotta una architettura **client-server**. Tale decisione è stata presa per diversi motivi:

- **Scalabilità:** L'architettura client-server consente di scalare facilmente il sistema, aggiungendo risorse al server senza dover modificare i client.
- **Manutenibilità:** Centralizzando la logica di business sul server, è più semplice effettuare aggiornamenti e manutenzioni.
- **Sicurezza:** La gestione centralizzata dei dati sul server permette di implementare misure di sicurezza più efficaci.
- **Accessibilità:** I client possono accedere al server da diverse piattaforme e dispositivi, migliorando la fruibilità del sistema.

I componenti principali dell'architettura sono il **client** che mette a disposizione una semplice interfaccia per gli utenti del sistema, e il **server** che gestisce la logica di business, l'accesso ai dati e le comunicazioni con i client. Il database viene utilizzato per la persistenza dei dati.

7.1.1 Client

Il Client è stato sviluppato usando **Java**, sfruttando il framework **JavaFX** per la realizzazione dell'interfaccia grafica. È stato scelto Java per la sua portabilità e per la vasta disponibilità di librerie che facilitano lo sviluppo di applicazioni desktop. JavaFX è stato selezionato per la sua capacità di creare interfacce utente moderne e reattive, oltre alla sua integrazione nativa con Java. Abbiamo scelto di adottare un'architettura Model-View-Controller (MVC) per il client, che separa la logica di presentazione dalla logica di business, migliorando la manutenibilità e la testabilità del codice. La scelta ricade su questo design in quanto il suo utilizzo presenta vantaggi in diversi aspetti:

- **Separazione delle responsabilità:** Ogni componente (Model, View, Controller) ha una responsabilità chiara e distinta, facilitando la gestione del codice.
- **Facilità di manutenzione:** Le modifiche alla logica di business o all'interfaccia utente possono essere effettuate in modo indipendente, riducendo il rischio di introdurre errori.
- **Testabilità:** La separazione delle componenti consente di testare ogni parte del sistema in modo isolato, migliorando la qualità del software.

7.1.2 Server

Il server è stato sviluppato utilizzando **Java** e gestito tramite **Maven** per la build e la gestione delle dipendenze. L'architettura backend si basa sulle specifiche **Jakarta EE**, in particolare utilizzando **Jakarta RESTful Web Services (JAX-RS)** per la creazione delle API REST. Questa struttura a più livelli consente di isolare le diverse responsabilità, migliorando la manutenibilità e la scalabilità del sistema. Il server comunica con un database **PostgreSQL** per la persistenza dei dati, utilizzando **JDBC** per l'interazione con il database.

Architettura backend

Il server adotta un'architettura a più livelli, suddividendo le responsabilità in vari strati:

- **Controller:** Gestisce le richieste HTTP in arrivo dai client, instradandole ai servizi appropriati.
- **Service:** Contiene la logica di business del sistema, elaborando i dati e coordinando le operazioni tra i vari componenti.
- **Data Access Object (DAO):** Si occupa dell'accesso ai dati, interagendo con il database per eseguire operazioni di lettura e scrittura.
- **Database Connection:** Gestisce la connessione al database, garantendo l'efficienza e la sicurezza delle operazioni di accesso ai dati.

7.2 Scelte tecnologiche adottate

In questa sezione vengono descritte le principali tecnologie e librerie utilizzate nello sviluppo del sistema, insieme alle motivazioni che hanno portato alla loro selezione.

7.2.1 Servizi Cloud usati:

- **Amazon EC2: Amazon Elastic Compute Cloud (EC2)** è un servizio di cloud computing che fornisce capacità di calcolo scalabile nel cloud gestito da Amazon Web Services (AWS). Consente di eseguire applicazioni su un'infrastruttura cloud sicura e affidabile, offrendo flessibilità nella gestione delle risorse di calcolo. Abbiamo scelto EC2 per la sua scalabilità, affidabilità e integrazione con altri servizi AWS.
- **Amazon S3: Amazon Simple Storage Service (S3)** è un servizio di storage di oggetti scalabile e durevole offerto da Amazon Web Services (AWS). Consente di archiviare e recuperare qualsiasi quantità di dati in qualsiasi momento, offrendo alta disponibilità e sicurezza. Abbiamo scelto S3 per garantire un'archiviazione sicura e scalabile dei file caricati dagli utenti e per la sua integrazione con altri servizi AWS.

8 Dockerfile e file di build automatica

In questa sezione vengono presentati il **Dockerfile** utilizzato per la creazione dell'immagine Docker del server e il processo di build automatica per configurare e distribuire l'applicazione in un ambiente containerizzato.

8.1 Dockerfile

Il file di configurazione per la creazione dell'immagine Docker adotta un approccio **Multi-Stage Build**. Questa strategia permette di separare l'ambiente di compilazione da quello di esecuzione, ottimizzando le dimensioni finali dell'immagine e migliorando la sicurezza.

Il processo è diviso in due stadi distinti:

1. **Stage 1: Build.** La prima fase utilizza l'immagine base `maven:3.8.5-openjdk-17`. In questo stadio vengono copiati il file `pom.xml` e il codice sorgente, ed eseguito il comando `mvn package` per generare l'artefatto JAR, escludendo i test unitari.
2. **Stage 2: Run.** La seconda fase utilizza un'immagine leggera `eclipse-temurin:17-jre`. Viene copiato solamente l'artefatto compilato dallo stage precedente e configurato l'entrypoint per avviare l'applicazione sulla porta 8080.

Di seguito è riportato il codice completo del Dockerfile utilizzato:

```
1 # Stage 1: Build
2 FROM maven:3.8.5-openjdk-17 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn dependency:go-offline
6 COPY src ./src
7 RUN mvn package -DskipTests
8
9 # Stage 2: Run
10 FROM eclipse-temurin:17-jre
11 WORKDIR /app
12
13 COPY --from=build /app/target/backend-1.0-SNAPSHOT.jar app.jar
14
15 EXPOSE 8080
16 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 1: Dockerfile per la build del backend

8.2 Orchestrazione con Docker Compose

Per facilitare la gestione dei container e la configurazione dell'ambiente, viene utilizzato lo strumento **Docker Compose**. Di seguito è mostrata la configurazione completa del file `docker-compose.yml`:

```
1 version '3.8'
2
3 services
4   backend
5     build .
6     container_name bugboard-backend
7     restart always
8     ports
9       - "8080:8080"
10    environment
11      - DB_URL=${DB_URL}
12      - DB_USER=${DB_USER}
13      - DB_PASSWORD=${DB_PASSWORD}
14      - JWT_SECRET=${JWT_SECRET}
15      - AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
16      - AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
17      - AWS_REGION=${AWS_REGION}
18      - AWS_BUCKET_NAME=${AWS_BUCKET_NAME}
19    depends_on
20      - db
21
22 db
23   image postgres13
24   container_name bugboard-db
25   restart always
26   environment
27     - POSTGRES_USER=${POSTGRES_USER}
28     - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
29     - POSTGRES_DB=${POSTGRES_DB}
30   ports
31     - "5432:5432"
32   volumes
```

```

33     - db-data/var/lib/postgresql/data
34
35 pgadmin
36     image dpape/pgadmin4
37     container_name bugboard-pgadmin
38     restart always
39     environment
40         # Credenziali per accedere all'interfaccia web di pgAdmin
41         PGADMIN_DEFAULT_EMAIL admin@example.com
42         PGADMIN_DEFAULT_PASSWORD Admin1926
43     ports
44         - "5050:80"
45     depends_on
46         - db
47
48 volumes
49     db-data

```

Listing 2: Configurazione docker-compose.yml

8.3 Descrizione dei Servizi Docker Compose

Il file `docker-compose.yml` configura l'architettura dell'applicazione definendo tre servizi principali che collaborano tra loro:

- **db:** Utilizza l'immagine ufficiale `postgres:13` per fornire un database relazionale robusto. È configurato con credenziali specifiche (utente, password, nome database) tramite variabili d'ambiente e utilizza un volume Docker (`db-data`) per garantire che i dati non vadano persi al riavvio dei container.

(Questo servizio gestisce la persistenza dei dati)

- **backend:** Definisce il servizio per l'applicazione principale, costruito in tempo reale utilizzando il `Dockerfile` presente nella directory corrente. Il servizio espone la porta `8080` per le richieste API e include tutte le configurazioni necessarie per la connessione al database, la sicurezza (JWT) e i servizi cloud (AWS). La direttiva `depends_on` assicura che questo container si avvii solo dopo il database.

(Questo servizio gestisce la logica di business dell'applicazione)

- **pgadmin:** Utilizza l'immagine `dpape/pgadmin4` per fornire un'interfaccia grafica web per la gestione del database PostgreSQL. Esposto sulla porta `5050`, permette agli amministratori

di interrogare e visualizzare i dati in modo visuale senza dover accedere alla riga di comando.
(Questo servizio fornisce gli strumenti di amministrazione e monitoraggio del database)

Questo approccio semplifica notevolmente la gestione dell'infrastruttura, consentendo un deployment replicabile e scalabile dell'intera applicazione con un singolo comando.

9 Design del Software

In questo capitolo viene approfondita la progettazione del sistema software realizzato. L'obiettivo è documentare le scelte implementative che hanno guidato la traduzione dei requisiti e dell'architettura in un software pienamente funzionante.

9.1 Schema per la persistenza dei dati

Il sistema utilizza un database relazionale (PostgreSQL) per la persistenza dei dati. Di seguito viene riportato lo schema logico delle tabelle definite.

9.1.1 Tabella User

```
1 CREATE TABLE User (  
2     username VARCHAR(100) PRIMARY KEY,  
3     email VARCHAR(100) NOT NULL UNIQUE,  
4     password_hash VARCHAR(100) NOT NULL,  
5     name VARCHAR(100) NOT NULL,  
6     surname VARCHAR(100) NOT NULL,  
7     role VARCHAR(100) NOT NULL,  
8     created_on TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT  
9         CURRENT_TIMESTAMP,  
10  
11     CONSTRAINT ck_email CHECK (email LIKE '_%@%._%'),  
12     CONSTRAINT ck_role CHECK (role IN ('Normal', 'Admin',  
13         'External'))  
14 );
```

Listing 3: Definizione della tabella User

9.1.2 Tabella Issue

```
1 CREATE TABLE Issue (  
2     issue_id SERIAL PRIMARY KEY,  
3     title VARCHAR(100) NOT NULL,  
4     description VARCHAR(1000) NOT NULL,  
5     type VARCHAR(100) NOT NULL,  
6     status VARCHAR(100) DEFAULT 'To Do' NOT NULL,  
7     created_on TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT  
8         CURRENT_TIMESTAMP,  
9     created_by VARCHAR(100),  
10  
11     CONSTRAINT ck_type CHECK (type IN ('Question', 'Documentation',  
12         'Bug', 'Feature')),  
13     CONSTRAINT ck_status CHECK (status IN ('To Do', 'In Progress',  
14         'Done', 'Archived')),  
15     CONSTRAINT fk_created_by FOREIGN KEY (created_by)  
16         REFERENCES User(username)  
17 );
```

Listing 4: Definizione della tabella Issue

9.1.3 Tabella Comment

```
1 CREATE TABLE Comment (  
2     comment_id SERIAL PRIMARY KEY,  
3     text VARCHAR(100) NOT NULL,  
4     created_on TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT  
5         CURRENT_TIMESTAMP,  
6     related_to INTEGER,  
7     written_by VARCHAR(100),  
8  
9     CONSTRAINT fk_comment_issue FOREIGN KEY (related_to)  
10        REFERENCES Issue(issue_id) ON DELETE CASCADE,  
11     CONSTRAINT fk_comment_user FOREIGN KEY (written_by)  
12        REFERENCES User(username)  
13 );
```

Listing 5: Definizione della tabella Comment

9.1.4 Tabella Change

```
1 CREATE TABLE Change (  
2     change_id SERIAL PRIMARY KEY,  
3     action VARCHAR(100) NOT NULL,  
4     details VARCHAR(1000),  
5     made_on TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT  
6         CURRENT_TIMESTAMP,  
7     created_by VARCHAR(100),  
8     related_to INTEGER,  
9  
10    CONSTRAINT fk_related_to FOREIGN KEY (related_to)  
11        REFERENCES Issue(issue_id) ON DELETE CASCADE,  
12    CONSTRAINT fk_created_by FOREIGN KEY (created_by)  
13        REFERENCES User(username)  
14 );
```

Listing 6: Definizione della tabella Change

9.1.5 Tabella Attachment

```
1 CREATE TABLE Attachment (  
2     attachment_id SERIAL PRIMARY KEY,  
3     file_name VARCHAR(100) NOT NULL,  
4     file_url VARCHAR(100) NOT NULL,  
5     uploaded_on TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT  
6         CURRENT_TIMESTAMP,  
7     uploaded_by VARCHAR(100),  
8     related_to INTEGER,  
9  
10    CONSTRAINT fk_related_to FOREIGN KEY (related_to)  
11        REFERENCES Issue(issue_id) ON DELETE CASCADE,  
12    CONSTRAINT fk_uploaded_by FOREIGN KEY (uploaded_by)  
13        REFERENCES User(username)  
14 );
```

Listing 7: Definizione della tabella Attachment

9.2 Diagramma delle classi

In questa sezione viene presentato il diagramma delle classi di design relative al modello di dominio del sistema. Il diagramma illustra le entità principali individuate durante l'analisi e la loro struttura interna (attributi e metodi principali), nonché le relazioni che intercorrono tra di esse. Le relazioni evidenziano come ogni elemento sia riconducibile a un utente creatore e, nel caso di commenti, allegati e modifiche, alla specifica segnalazione di riferimento.



Figura 20: Diagramma delle classi del back-end



Figura 21: Diagramma delle classi del front-end

9.3 Scelte di software design

In questa sezione vengono illustrate le principali decisioni progettuali che hanno guidato lo sviluppo del codice sorgente. L'approccio adottato mira a garantire la qualità del software in termini di modularità, manutenibilità ed estensibilità. La progettazione delle classi e delle loro interazioni è stata realizzata seguendo i paradigmi della programmazione orientata agli oggetti, con una forte enfasi sulla separazione delle responsabilità e sul disaccoppiamento dei componenti. Di seguito vengono approfonditi i pattern specifici e i principi che costituiscono le fondamenta logiche dell'applicazione.

9.3.1 Design Pattern adottati

Durante lo sviluppo sono stati applicati diversi design pattern per risolvere problemi ricorrenti:

- **Singleton Pattern:** Utilizzato per le classi di servizio, i DAO e la gestione della connessione al database. Questo garantisce che esista una sola istanza di questi componenti stateless in memoria, ottimizzando l'uso delle risorse.
- **DAO Pattern:** Separa la logica di accesso ai dati dalla logica di business, permettendo di modificare l'implementazione della persistenza senza impattare sui livelli superiori.
- **DTO Pattern:** Utilizzato per trasferire dati tra il client e il server, evitando di esporre direttamente le entità del database e permettendo di aggregare o filtrare i dati inviati in risposta.

9.3.2 Adesione ai principi SOLID

Il design del software è stato guidato dai principi SOLID per garantire modularità e manutenibilità:

- **Single Responsibility Principle (SRP):** Ogni classe ha una responsabilità ben definita. Ad esempio, i `Controller` gestiscono solo le richieste HTTP, i `Service` contengono la logica di business e i `DAO` si occupano esclusivamente dell'accesso ai dati. Anche le classi di utilità come `DatabaseConnection` hanno un unico scopo (gestire la connessione).
- **Open/Closed Principle (OCP):** Il sistema è aperto all'estensione ma chiuso alla modifica. L'uso di interfacce per i DAO permette di aggiungere nuove implementazioni di persistenza (es. un database diverso o un mock per i test) senza modificare il codice che le utilizza nei `Service`.

- **Liskov Substitution Principle (LSP):** Le implementazioni dei DAO rispettano i contratti definiti dalle loro interfacce, garantendo che possano essere sostituite senza alterare la correttezza del programma.
- **Interface Segregation Principle (ISP):** Le interfacce dei DAO sono granulari e specifiche per entità, evitando di costringere le classi a dipendere da metodi che non utilizzano.
- **Dependency Inversion Principle (DIP):** I componenti di alto livello (Service) non dipendono direttamente dalle implementazioni di basso livello (DAO concreti), ma dalle loro astrazioni (Interfacce). L'uso delle interfacce per i componenti di basso livello disaccoppia la logica di business dall'implementazione specifica della persistenza.

10 Strumenti e strategie di versioning

10.1 Versioning System

Come versioning system è stato utilizzato Git con hosting del repository su GitHub. Git è uno strumento che permette la collaborazione tra membri di un team che lavorano sul progetto tramite modifiche effettuate in locale, mediante clonazione del codice sorgente da parte di ognuno dei collaboratori, e una successiva congiunzione del lavoro sul repository remoto.

Repository su GitHub: https://github.com/xFireFox27/BugBoard26-INGSW2526_015

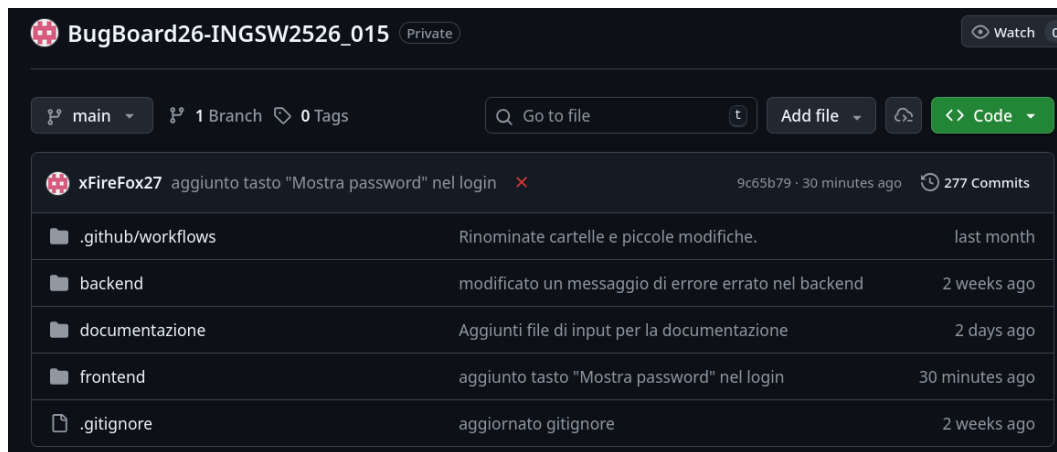


Figura 22: Screenshoot del repository su GitHub

10.1.1 Branching Strategy

Come branching strategy è stato utilizzato un "Centralized Workflow", una strategia che prevede che tutti i contributors lavorino direttamente sull'unico branch principale chiamato main. Questa strategia è quella che più facilmente si sposa con un gruppo ridotto di contributors a stretto contatto tra loro. In particolare le iterazioni prevedevano una comunicazione frequente tra i vari membri del team, con una divisione strategica dei task volta ad evitare overlap delle modifiche, limitando così errori derivanti dal merging, e l'incoraggiamento di commit frequenti per facilitare l'integrazione del codice e la rapida risoluzione di eventuali errori derivanti da conflitti. Questo tipo di strategia è perfetta per processi di sviluppo rapidi dato che taglia l'overhead derivante dalla gestione dei branch separati.

11 Statistiche Repository

11.1 Premessa

Per trasparenza e concretezza dei dati riportati questi fanno riferimento ai periodi di attività del team, non saranno quindi considerati nei calcoli il periodo compreso tra il 19 Ottobre e il 15 Novembre e la settimana del 28 Settembre, nella quale è stato creato il repository, ma che non ha visto ulteriori svolgimenti. Inoltre è bene notare che il primo dei due periodi di attività fa riferimento esclusivamente all'inizio dell'analisi della consegna e della stesura delle documentazione con Glossario¹, Analisi dei Requisiti^{3,4}, Scrittura degli use case diagrams², Formalizzazione dei casi d'uso⁵ e creazione di Personas⁶; quindi durante questa prima fase non sono stati prodotti artefatti software. I dati sono aggiornati al giorno 26/12/2025

11.2 Analisi dei Contributors

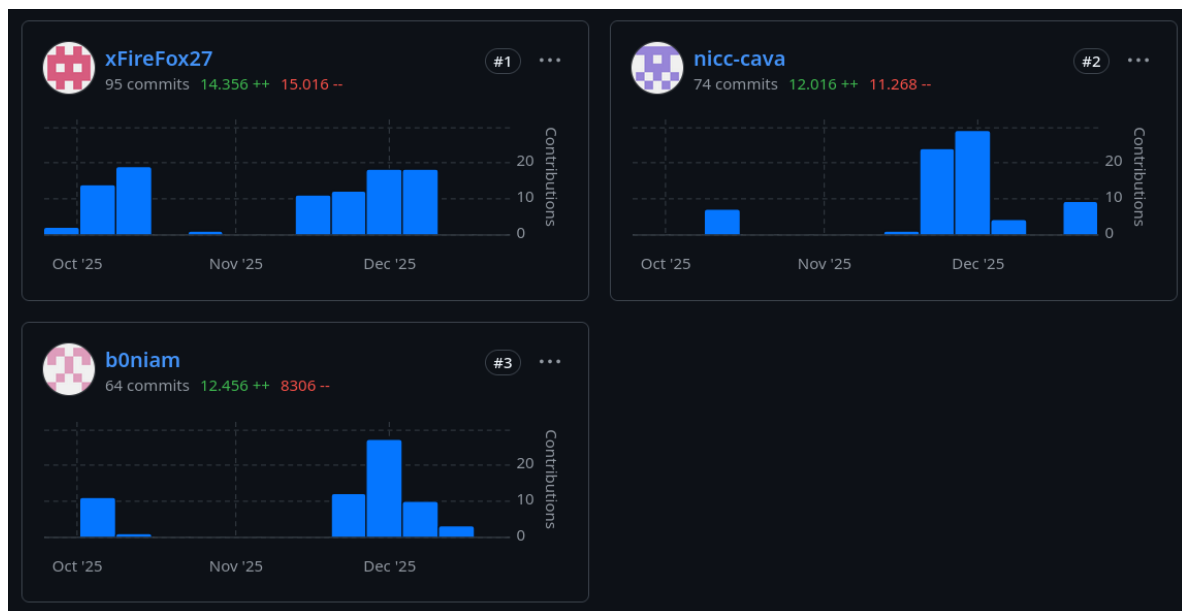


Figura 23: Screenshot dei Contributors, da GitHub

Come evincibile dalla schermata dei contributors²³ il lavoro è stato diviso piuttosto equamente con una media di circa 76 commit a contributore, 12.189 linee di codice inserite e 11.326 linee di codice eliminate.

11.3 Analisi dei Commit

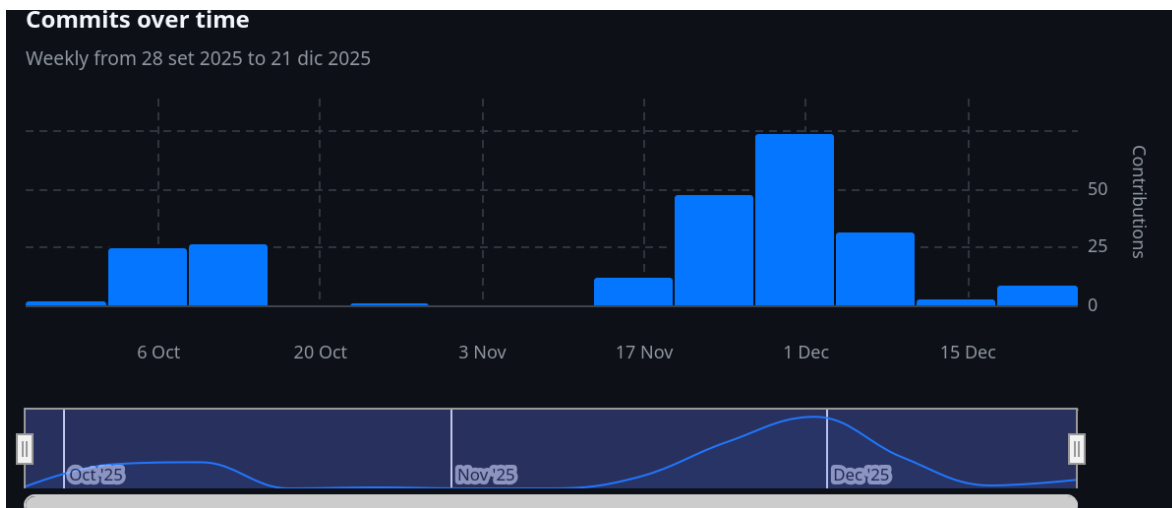


Figura 24: Screenshot dei Commits over time, da GitHub

Il repository, nei periodi di attività sopra indicati^{11.1}, ha visto 232 commits con una media di 29 commits a settimana durante i due distinti periodi. Facendo riferimento ai singoli periodi, invece, abbiamo un totale di 52 commit e una media settimanale di 26 commit per il primo e 180 totali con una media di 30 commit a settimana per il secondo.

11.4 Analisi delle Linee di Codice



Figura 25: Screenshot della "Code Frequency", da GitHub

Facendo riferimento ai periodi di attività sopra citati^{11.1}, il repository ha visto l'inserimento di 36.569 linee di codice con una media di 4.571 a settimana, delle quali 16.739 con circa 8.370 di media settimanale sono state inserite nel primo dei due periodi mentre 19.830 inserimenti totali con media settimanale di 3.305 nel secondo, e la rimozione di 33.978 con una media di 4.247 rimozioni per settimana divise in 20.554 nel primo periodo con una media di 10.277 settimanale e le restanti 13.424 nel secondo con una media settimanale di 2.237 rimozioni.

12 Report di Qualità del Codice

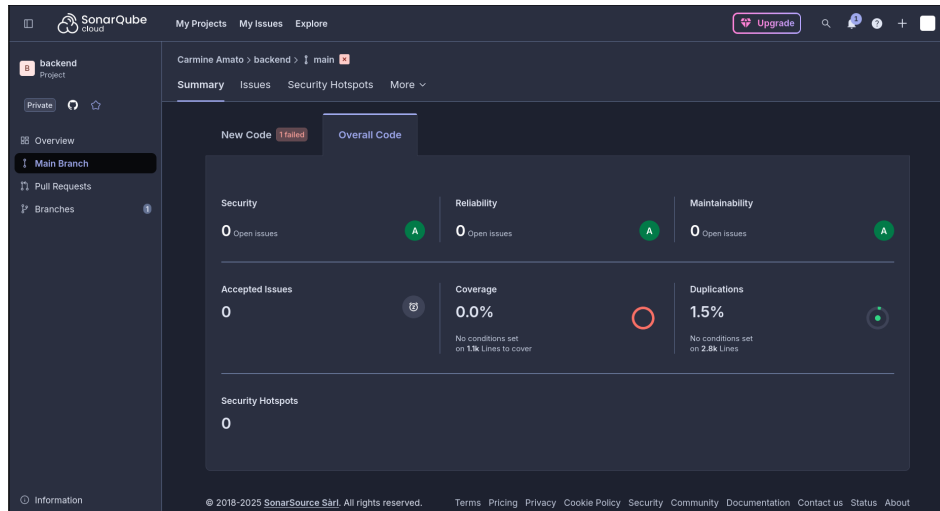


Figura 26: Report di Qualità da Sonar Qube Cloud

Per avere costante contezza della qualità del codice scritto, ci siamo serviti di SonarQube Cloud e SonarQube (attraverso estensione per IDE).

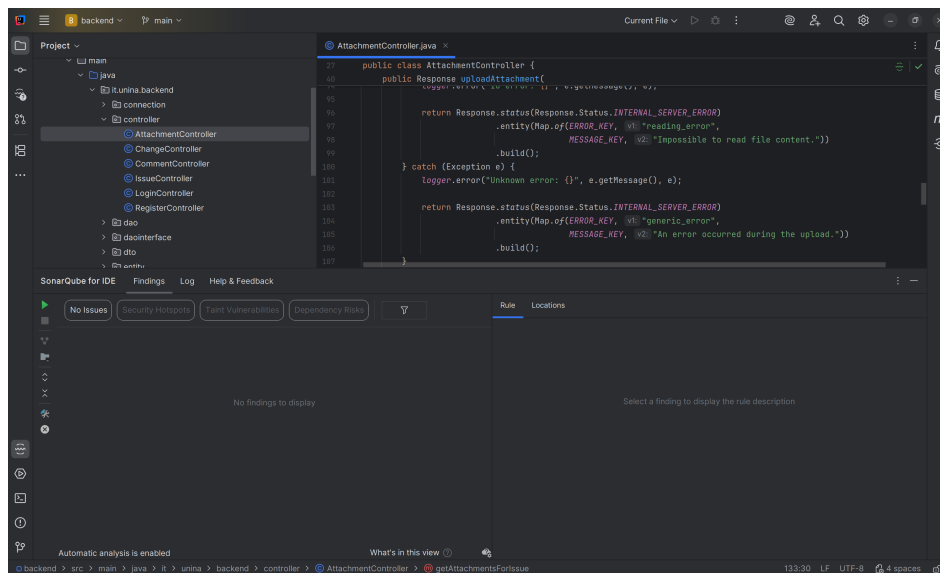


Figura 27: Esempio di Utilizzo di SonarQube for IDE su IntelliJ IDEA